Composite

- Definition
- The Problem
- Solution: Composite Design Pattern
  - Use cases
  - Class Diagram
  - Structure of Composite Pattern
- Implementation
  - Example 1: File Structure
  - Example 2: Arithmetic Expressions

Resources

- Video → 32. All Structural Design Patterns | Decorator, Proxy, Composite, Adapter, Bridge, Facade, FlyWeight
- Video → 19. Design File System using Composite Design Pattern | Low Level Design Interview Question | LLD

Definition

> ***The Composite Design Pattern*** *allows you to compose objects into tree structures to* ***represent a part-whole hierarchies*** *which lets* ***clients*** *manage and perform operations on* ***individual objects(leaf nodes) and compositions of objects(composite nodes) uniformly*** *(treating both objects the same way i.e., without knowing whether it's working with a component or a composite).*

Concept && Coding

The Problem

Understanding the problems by implementing the File Structure example using a naive approach:

```java
public class File {
    String fileName;

    public File(String name) {
        this.fileName = name;
    }

    public void printContents() {
        System.out.println("File name: " + fileName);
    }
}
```

```java
public class Directory {
    String directoryName;
    List<Object> objectList;

    public Directory(String name) {
        this.directoryName = name;
        objectList = new ArrayList<>();
    }

    public void add(Object object) {
        objectList.add(object);
    }
```

```java

    public void remove(Object object) {
        objectList.remove(object);
    }

    // Display full structure
    // Breaks OCP - if we want to add a new file type, we need to modify this method to add another if/else condition
    public void printContents() {
        System.out.println("Directory Name: " + directoryName);
        for (Object obj: objectList) {
            if (obj instanceof File) {
                ((File) obj).printContents();
            } else if (obj instanceof Directory) {
                ((Directory) obj).printContents();
            }
        }
    }
}
```

```java
// Client Code
public class Client {
    public static void main(String[] args) {
        Directory movieDirectory = new Directory("Movies");

        File rentalReceipt = new File("RentalReceipt");
        movieDirectory.add(rentalReceipt);

        Directory comedyMovieDirectory = new Directory("ComedyMovies");
        File dumbAndDumber = new File("DumbAndDumber");
        comedyMovieDirectory.add(dumbAndDumber);
        movieDirectory.add(comedyMovieDirectory);

        movieDirectory.printContents();
    }
}
```

Concept && Coding

- **No Common Abstraction**
  - `File` and `Directory` → Both are different types.
  - If you write client code, you need to know whether **it is a File or a Folder** before calling methods.

```java
public void printContents() {
    System.out.println("Directory Name: " + directoryName);
    for (Object obj : objectList) {
        if (obj instanceof File) {
            ((File) obj).printContents();
        } else if (obj instanceof Directory) {
            ((Directory) obj).printContents();
        }
    }
}
```

  - This breaks the Open/Closed Principle. If we want to add a new file type, we need to modify this method to add another `if/else` condition(which is messy).
- **Scalability Issue**
  - If later you add a new type (say a `CompressedFolder`), you have to update everywhere you wrote those `if-else` checks.
  - This makes the system rigid, tightly-coupled, and hard to extend.

Solution: Composite Design Pattern

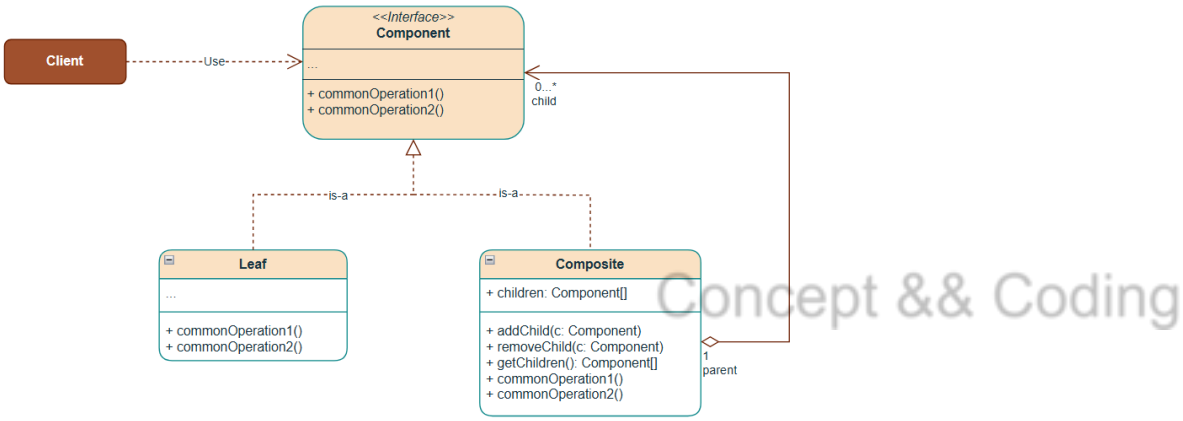
With the Composite Pattern:

- Both `File` and `Folder` implement the same Component interface.
- Client code doesn't have to know whether it's a file or a folder, and hence it can treat both as composite and perform operations uniformly(advantage of using abstraction).
- Adding new types (e.g., `Shortcut` or `ZipFolder`) doesn't require rewriting client logic, making it an extensible design.

Use cases

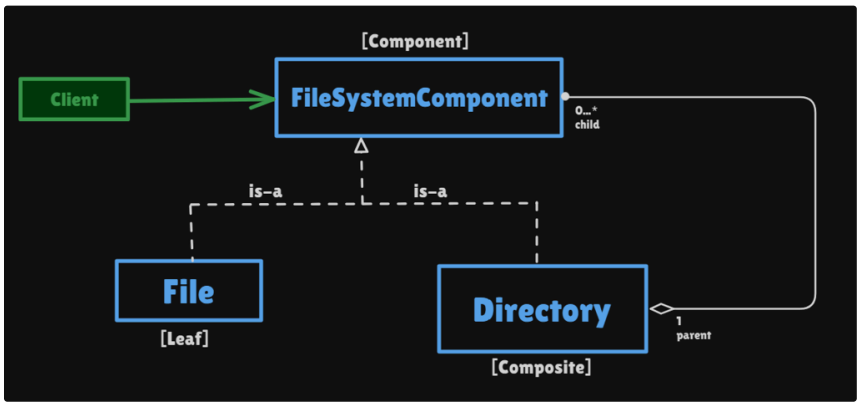
1. File Systems
2. Organizational Structures
3. Mathematical Expressions
4. Building Structures
5. Drawing applications and Computer-Aided Design (CAD)

Class Diagram



Structure of Composite Pattern

Understanding the Structure of Composite Pattern using File Structure Example:



- **Component (`FileSystemComponent`)**: Abstract base class/Interface defines operations common to both simple (leaf) and complex (composite) objects.
- **Leaf (`File`)**: Represents a leaf node(end objects) in the composition. A leaf has no children and implements the component interface directly.

- **Composite (`Directory`)**: Defines the behavior for composites(components that can have children).
- **Client (`FileSystemDemo`)**: Works with both component and composite objects uniformly through the Component interface.

Implementation

Example 1: File Structure

```java
// Step 1: Component interface
public interface FileSystemComponent {
    void printContents();
}
```

```java
// Step 2: Leaf - File
public class File implements FileSystemComponent {
    String fileName;

    public File(String name) {
        this.fileName = name;
    }

    @Override
    public void printContents() {
        System.out.println("File name: " + fileName);
    }
}
```

```java
// Step 3: Composite - Folder
public class Directory implements FileSystemComponent {
    String directoryName;
    List<FileSystemComponent> children;

    public Directory(String name) {
        this.directoryName = name;
        children = new ArrayList<>();
    }

    public void add(FileSystemComponent fileSystemComponent) {
        children.add(fileSystemComponent);
    }

    public void remove(FileSystemComponent fileSystemComponent) {
        children.remove(fileSystemComponent);
    }

    @Override
    public void printContents() {
        System.out.println("Directory Name: " + directoryName);
        for (FileSystemComponent child : children) {
            child.printContents();
        }
    }
}
```

Concept && Coding

```java
// Step 4: Client code with Composite Pattern
public class FileSystemDemo {
    public static void main(String[] args) {
        System.out.println("======= Composite Design Pattern ======");

        // Create files
        File receipt = new File("receipt.pdf");
        File invoice = new File("invoice.pdf");
        File torrentLinks = new File("torrentLinks.txt");
        File tomCruise = new File("tomCruise.jpg");
        File dumbAndDumber = new File("DumbAndDumber.mp4");
        File hangoverI = new File("HangoverI.mp4");
```

```java

        // Create directories
        Directory moviesDirectory = new Directory("Movies");
        Directory comedyMovieDirectory = new Directory("ComedyMovies");

        // Build the tree structure hierarchically
        moviesDirectory.add(receipt);
        moviesDirectory.add(invoice);
        moviesDirectory.add(torrentLinks);
        moviesDirectory.add(tomCruise);
        moviesDirectory.add(comedyMovieDirectory);
        comedyMovieDirectory.add(dumbAndDumber);
        comedyMovieDirectory.add(hangoverI);

        // Display full structure
        moviesDirectory.printContents();
    }
}
```

Example 2: Arithmetic Expressions

```java
// Component Interface
public interface ArithmeticExpression {
    int evaluate();
}
```

```java
// Leaf
public class Number implements ArithmeticExpression {
    int value;

    public Number(int value) {
        this.value = value;
    }

    public int evaluate() {
        System.out.println("Number value is: " + value);
        return value;
    }
}
```

Concept && Coding

```java
// Composite
public class Expression implements ArithmeticExpression {

    ArithmeticExpression leftExpression;
    ArithmeticExpression rightExpression;
    OperationType operation;

    public Expression(ArithmeticExpression leftPart, ArithmeticExpression rightPart, OperationType operation) {
        this.leftExpression = leftPart;
        this.rightExpression = rightPart;
        this.operation = operation;
    }

    public int evaluate() {
        int value = 0;
        switch (operation) {
            case OperationType.ADD:
                value = leftExpression.evaluate() + rightExpression.evaluate();
                break;
            case OperationType.SUBTRACT:
                value = leftExpression.evaluate() - rightExpression.evaluate();
                break;
            case OperationType.DIVIDE:
                value = leftExpression.evaluate() / rightExpression.evaluate();
                break;
```

```java
            case OperationType.MULTIPLY:
                value = leftExpression.evaluate() * rightExpression.evaluate();
                break;
        }

        System.out.println("Expression value is:" + value);
        return value;
    }
}
```

```java
// Client Code with Composite Pattern
public class MathExpressionEvaluator {
    public static void main(String[] args) {
        System.out.println("======= Composite Design Pattern ======");
        // 2*(1+7) tree structure for evaluation
       /*
                        *
                      /   \
                    2      +
                          / \
                         1   7
        */
        ArithmeticExpression two = new Number(2);
        ArithmeticExpression one = new Number(1);
        ArithmeticExpression seven = new Number(7);

        ArithmeticExpression addExpression = new Expression(one, seven, OperationType.ADD);
        ArithmeticExpression parentExpression = new Expression(two, addExpression, OperationType.MULTIPLY);

        System.out.println(parentExpression.evaluate());
    }
}
```

Concept && Coding

```java
public enum OperationType {
    ADD, SUBTRACT, MULTIPLY, DIVIDE
}
```